

軟體品質與測試

第 1 章：軟體危機、品質模型與 AI 時代的可靠性工程

授課教師：薛念林教授 (with Gemini AI)

本章重點 (Key Topics)

- **1.1 軟體危機的歷史與 AI 時代的輪迴**：愛國者、NASA、華航、迪士尼重大案例與軟體危機成因。
- **1.2 AI 能拯救軟體危機嗎？**：AI 生成代碼的維護債、高錯誤率、安全弱點與四大典型品質事故。
- **1.3 軟體的本質與品質維度**：IEEE 軟體四要素、David Garvin 五大品質觀點。
- **1.4 軟體品質工程核心概念**：V&V 驗證與確認、CoQ 品質成本與 1:10:100 定律、測試左移。
- **1.5 生命週期品質把關**：V 模型對稱性、DevOps CI/CD 六大連續品質門檻。
- **1.6 現代軟體品質模型**：ISO 25010 八大品質特性與 16 週測試技術地圖。

1.1 軟體危機的歷史與 AI 時代的輪迴

大家都知道「物質不滅定律」；身為資工系學生，我們更熟悉「Bug 不滅定律」。

1.1.1 Case 1：愛國者反導彈事件 (1991)

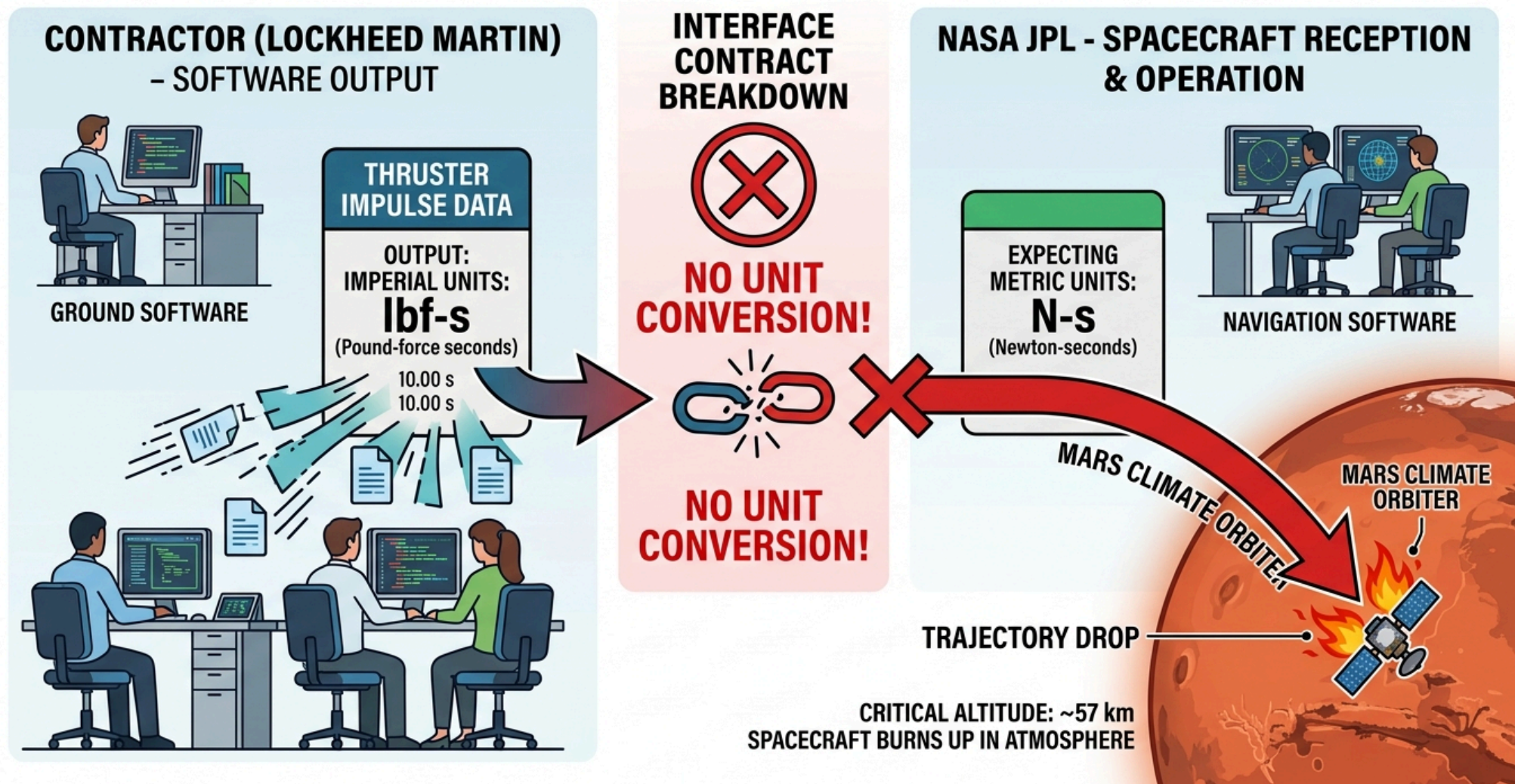
- 事件背景：
 - 1991 年波斯灣戰爭，伊拉克飛毛腿飛彈擊中美軍沙烏地達蘭基地，造成 **28 名美軍死亡、100+ 人受傷**。
- 致命軟體缺陷：
 - 愛國者系統時鐘暫存器採用 **24-bit 浮點數** 設計，將時間轉為 0.1 秒單位時產生截斷誤差（約 0.000000095 秒）。
 - 系統連續開機運作超過 **100 小時** 未重啟，誤差累計達 **0.33 秒**。
- 災難後果：
 - 飛毛腿飛彈速度達 4.2 馬赫（1.5 km/s），0.33 秒相當於 **600 公尺距離偏差**，雷達搜尋窗無法鎖定目標，攔截飛彈未發射。
- **SQA 啟示**：數值精度問題、浮點數累計誤差，以及**長時運行可靠度測試（Long-term Reliability Testing）**的重要性。

1.1.2 Case 2：NASA 火星氣候軌道探測器 (1998)

- 事件背景：
 - 1998 年 NASA 發射「火星氣候軌道探測器」（造價近 2 億美元），抵達火星後失聯焚毀。
- 致命缺陷：跨模組單位不一致
 - 承包商端（洛克希德馬丁）：地面程式以 英制單位（磅力·秒，**lbf·s**）輸出推進器衝量。
 - NASA JPL 導航接收端：太空船軟體預設以 公制單位（牛頓·秒，**N·s**）解析數據（相差 4.45 倍）。
- 災難後果：
 - 軌道高度預計 140 公里，實際暴跌至 **57 公里**，直接在大氣層中摩擦解體。
- SQA 啟示：跨模組介面契約（**Interface Contract**）、強型態檢驗與規格審查的重要性。

Case 2 架構圖解：跨模組介面契約斷裂

1998 THE NASA MARS CLIMATE ORBITER SOFTWARE/ SOFTWARE FAILURE DUE TO UNIT MISMATCH



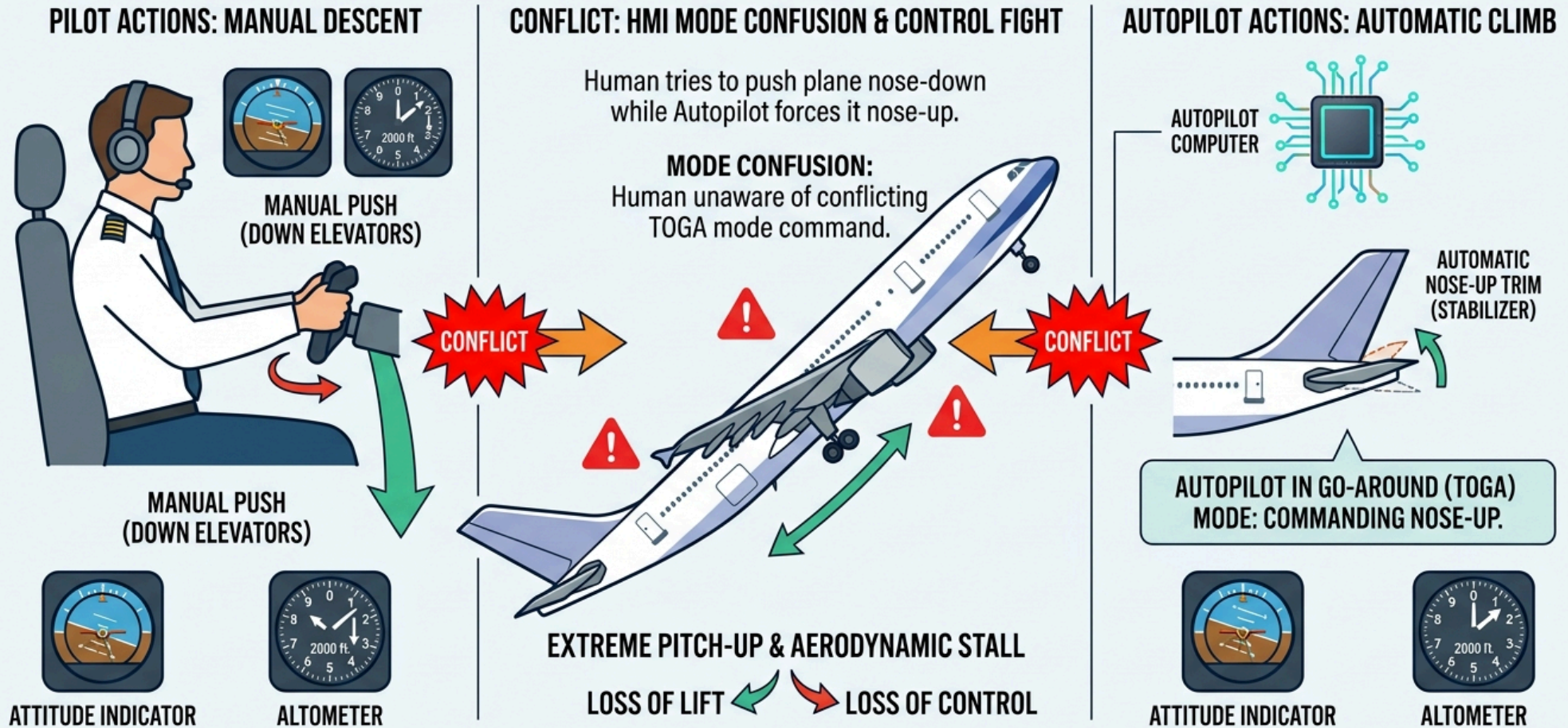
1.1.3 Case 3：華航名古屋空難 (1994)

- 事件背景：
 - 1994 年華航 CI140 班機 (A300-622R) 在名古屋機場降落時墜毀，**264 人罹難**。
- 人機介面衝突 (HMI Mode Confusion)：
 - **機師手動操作 (Manual Push)**：副駕駛誤觸重飛模式後，正副駕駛試圖手動強推操縱桿強壓機首降落。
 - **飛控電腦自動配平 (Autopilot Climb)**：電腦處於重飛狀態，強行將水平安定面向上配平抬高機首。
- 致命後果：
 - 駕駛員未察覺電腦仍在執行重飛，人機相互抵消；最終達到極限仰角，飛機在低空氣動失速 (**Stall**) 墜毀。
- **SQA 啟示**：人機互動 (HMI/UX) 狀態透明度、異常操作回饋與自動化控制權仲裁。

Case 3 架構圖解：人機介面衝突與控制權仲裁

1994 CHINA AIRLINES NAGOYA CRASH: FLIGHT CONTROL SOFTWARE CONFLICT (HMI MODE CONFUSION)

Case Study: Human-Machine Interface Design & System Safety



1.1.4 Case 4：迪士尼《獅子王》遊戲 (1994)

- **事件背景：**

- 1994 年聖誕節迪士尼推出《獅子王》PC 遊戲，數以萬計家庭滿心期待安裝同樂。

- **致命缺陷：缺乏相容性測試**

- 遊戲基於特定視訊驅動（WinG）開發，**未在市場主流多樣硬體環境上進行充分相容性測試。**

- **災難後果：**

- 大量家用電腦開機即藍屏當機，客服專線被憤怒家長打爆，嚴重重創品牌聲譽。

- **SQA 啟示：**

- 環境多樣性驗證與**相容性測試（Compatibility Testing）**的重要性。
- 該事件促使微軟後來開發標準化 DirectX 遊戲架構。

1.1.5 軟體危機的定義與成因

- 1968 年 NATO 會議首次提出「軟體危機 (Software Crisis)」：

1. 軟體規模與複雜性失控：

- 硬體性能激增，軟體規模呈指數成長，超出傳統管理極限。

2. 軟體開發效率低下：

- 進度與成本難以預測，人月神話加劇溝通成本。

3.

軟體品質低下：

- 錯誤率高，缺乏系統化品質驗證手段。

4.

軟體維護困難：

- 架構腐化、缺乏文件，後期維護成本吞噬所有研發預算。

CCQ 1 - 概念核對問答

愛國者反導彈系統（1991）在達蘭基地攔截失效的根本軟體原因為何？

- **A.** 通訊網路中斷導致雷達無法傳送指令給飛彈發射架
- **B.** 24-bit 時鐘暫存器的浮點捨入誤差在連續運行 100 小時後累加達 0.33 秒
- **C.** 程式碼發生記憶體洩漏（Memory Leak）導致作業系統當機
- **D.** 雷達演算法誤將美軍戰機辨識為敵方飛毛腿飛彈



1.2 AI 能拯救軟體危機嗎？

在 2026 年，寫出一段程式碼只要問 AI 3 秒鐘；
但要證明這段程式碼不會搞垮公司，可能要花上 3 個月。

1.2 AI 輔助開發的實證研究數據

- **1. 程式碼維護性惡化 (GitClear 1.5 億行研究, 2020-2026)：**
 - 程式碼重複率 呈指數級上升。
 - 重構指標 「移動行數 (Moved Lines)」大幅下降，工程師更少主動重構。
 - 程式碼流失率 (Churn) 顯著增高，帶來沈重的長期維護性債務。
- **2. 52% 高錯誤率與「虛假安全感」 (Purdue University)：**
 - ChatGPT 解答 Stack Overflow 問題時，**52%** 包含錯誤程式碼或資訊。
 - 因 AI 語氣自信且條理分明，**39.3%** 的使用者依然採信了 AI 的錯誤回答。
- **3. 40% 安全弱點隱患 (NYU 等學術研究)：**
 - 在無安全提示引導下，AI 生成程式碼中有 約 **40%** 包含 **CWE 安全漏洞**（如緩衝區溢位、SQL 注入）。

1.2.1 AI 寫程式引發的品質事件 (1/2)

- **1. 幻覺套件供應鏈投毒 (Slopsquatting / Package Hallucination) :**
 - 機制：LLM 憑空捏造合理套件名稱 (如 `crypto-validator`)。
 - 災難：黑客搶先在 PyPI/npm 註冊同名惡意套件，工程師直接執行 `pip install` 導致後門植入企業生產環境。
- **2. 亞馬遜 (Amazon) 電商系統大斷線與訂單蒸發 :**
 - 機制：2026 年 3 月工程師使用 AI 寫程式工具生成代碼變更，未經充分審查即推上生產環境。
 - 災難：送貨與結帳邏輯出錯，北美訂單一度崩跌 99%，數小時內蒸發超過 630 萬筆訂單與鉅額營收。

1.2.1 AI 寫程式引發的品質事件 (2/2)

- **3. Vibe Coding 帶來的漏洞大爆發（以 Lovable/No-code 平台為例）：**
 - 機制：非工程人員憑 Prompt 快速產出 Web 服務，缺乏 Code Review 與安全概念。
 - 災難：抽查 1,600+ 個上線應用，高達 10%+ 存在嚴重 SQLi 或越權存取（BOLA）漏洞，可直接繞過驗證進後台。
- **4. 敏感金鑰與憑證直接寫死（Hardcoded Secrets）外洩：**
 - 機制：AI 範例常把 API Key、資料庫密碼寫死在代碼中。
 - 災難：開發者直接推送到公開 GitHub Repo，雲端帳號 1 小時內被爬蟲盜用並產生數萬美元帳單。

AI 時代軟體本質：從 Writing 轉移到 Verification

‘Software and cathedrals are much the same – first we build them, then we pray.’ — Sam Redwine



1.3 軟體的本質與品質維度

軟體四要素 & David Garvin 五大品質觀點

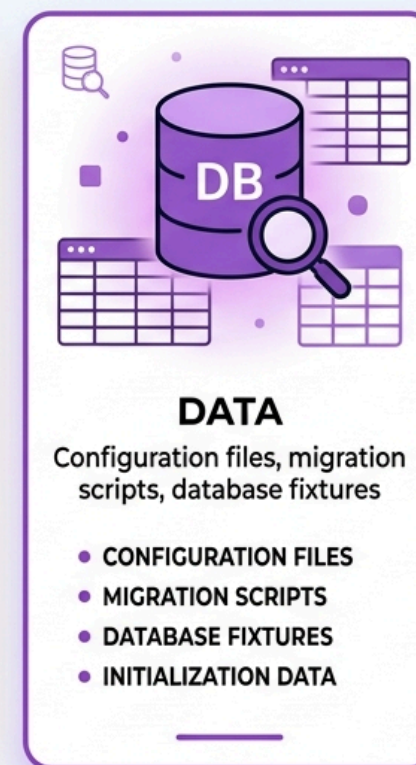
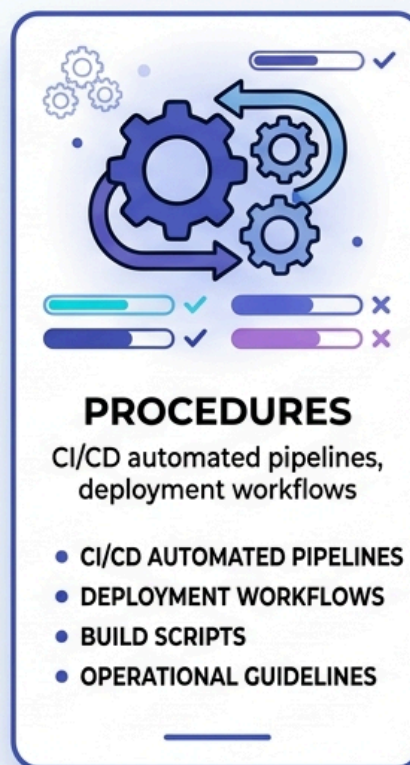
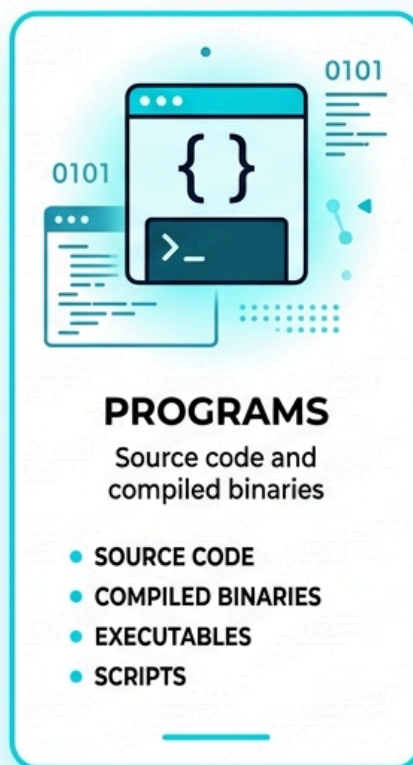
1.3.1 軟體四大核心要素 (IEEE 610.12)

Software (軟體): *Computer programs, procedures, and possibly associated documentation and data.*

- **1. Programs (程式碼)**：原始碼、編譯 Bytecode 與執行腳本，承載業務邏輯。
- **2. Procedures (作業程序)**：CI/CD 構建腳本、部署規程與維運手冊 (Runbooks)。
- **3. Documentation (文件與規格)**：需求規格書 (SRS)、OpenAPI 介面契約與測試計畫。
- **4. Data (資料與配置)**：資料庫結構 (Migration)、環境配置與測試測資集 (Test Fixtures)。

軟體四大核心要素架構圖 (IEEE 610.12)

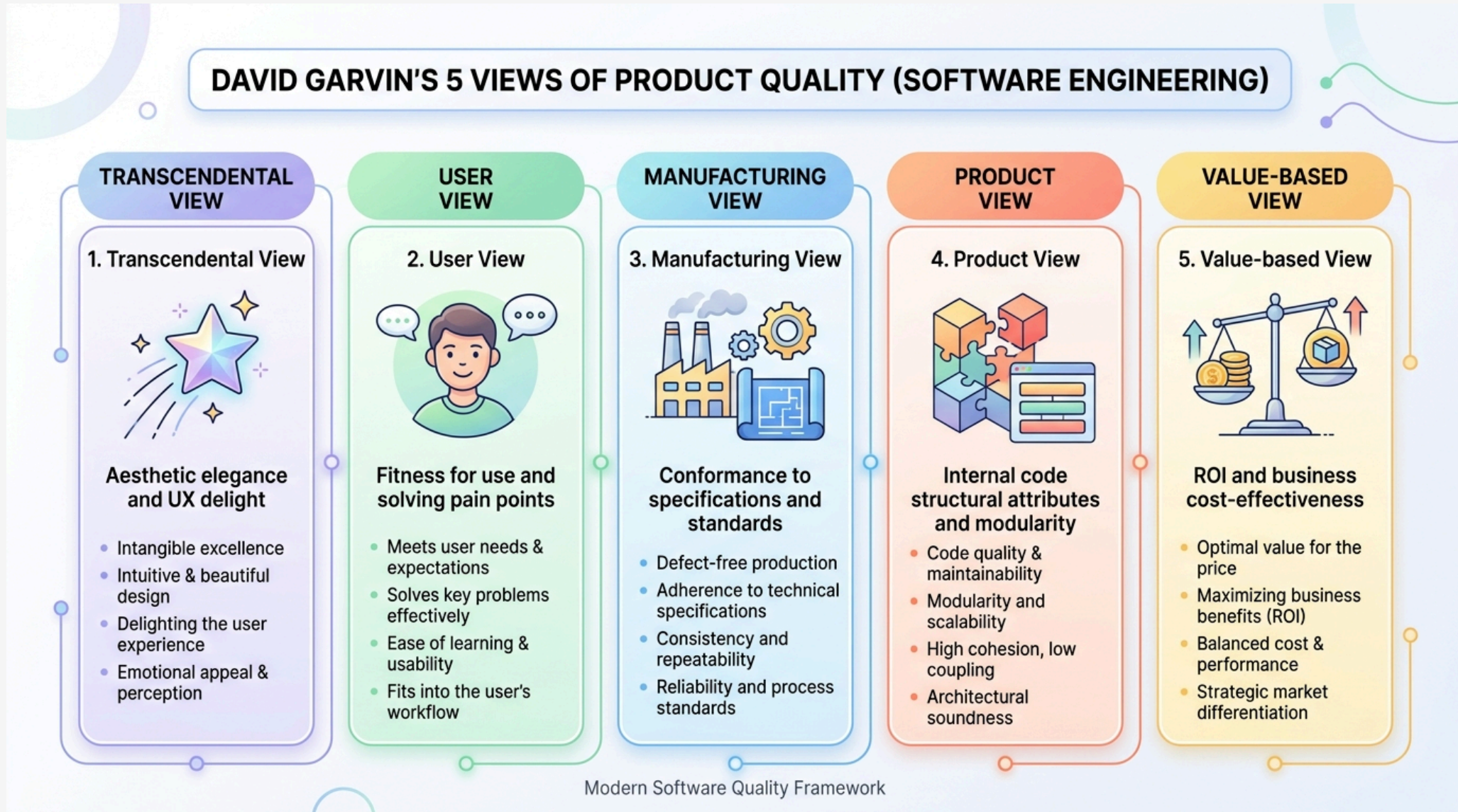
EDUCATIONAL SOFTWARE ENGINEERING INFOGRAPHIC: 4 CORE ELEMENTS OF SOFTWARE (IEEE STANDARD)



1.3.2 David Garvin 五大品質觀點

- **1. 超自然觀點 (Transcendental View) :**
 - 無法量化，但一體驗就能感受其極致優雅與美感（如流暢的 UI/UX 微互動）。
- **2. 使用者觀點 (User View - Fitness for Use) :**
 - 是否切中使用者痛點、操作直覺並帶來實質效益（合用性）。
- **3. 製造觀點 (Manufacturing View - Conformance) :**
 - 是否 100% 符合工程規格書、通過靜態檢測與 Quality Gate。
- **4. 產品觀點 (Product View - Architecture) :**
 - 產品內在技術特性，如高內聚低耦合、強固型態、可測試性與可維護性。
- **5. 價值觀點 (Value-based View - ROI) :**
 - 軟體商業效益是否顯著高於開發、測試與維運之總成本。

David Garvin 五大品質觀點架構



Garvin 五大品質觀點對照表

品質觀點	核心定義	軟體工程實例	忽略該觀點的後果
超自然觀點	無法量化，體驗感受極致美感	流暢 UI/UX、細膩微互動	軟體感覺粗製濫造難用
使用者觀點	符合真實需求 (Fitness for Use)	解決痛點、操作直覺	做出來沒人想用 (Shelfware)
製造觀點	符合規格流程 (Conformance)	遵循 Clean Code、通過 Gate	規格有漏洞時做出一套完美垃圾
產品觀點	產品內在技術特性與架構	高內聚低耦合、強固型態	架構腐化，改功能引發崩潰
價值觀點	商業價值與性價比 (ROI)	商業產出 > 開發維運成本	開發成本失控，商業不可行

CCQ 2 - 概念核對問答

某專案團隊開發的電商 App 完全符合合約規格書上的每一條需求（製造觀點合格），但因底層架構高度耦合且無單元測試，半年後想新增促銷功能時必須重寫整個系統。這代表在 Garvin 哪一個品質觀點嚴重不及格？

- **A.** 產品觀點 (Product View)
- **B.** 製造觀點 (Manufacturing View)
- **C.** 法律合約觀點
- **D.** 超自然觀點 (Transcendental View)



1.4 軟體品質工程核心概念

V&V 驗證與確認、品質成本 (CoQ) 與測試左移

1.4.1 驗證與確認 (Verification vs. Validation)

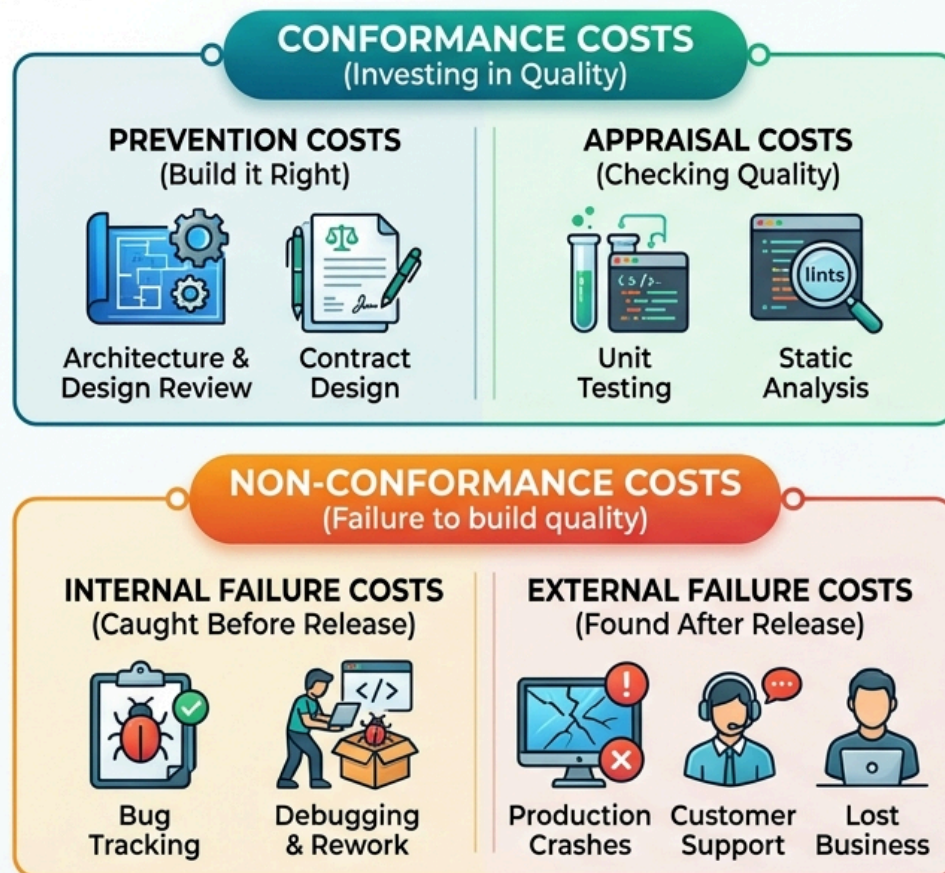
- 🔍 **Verification (驗證) :**
 - *Are we building the product **right**?* (我們是否有正確地建造軟體?)
 - 確保產出物符合設定的規格 (如程式碼是否符合設計圖、單元測試是否符合規格)。
- 🎯 **Validation (確認) :**
 - *Are we building the **right** product?* (我們建造的是否是正確的軟體?)
 - 確保軟體真正滿足使用者的真實業務需求 (驗收測試、易用性測試、Beta 測試)。

1.4.2 軟體品質成本 (Cost of Quality, CoQ)

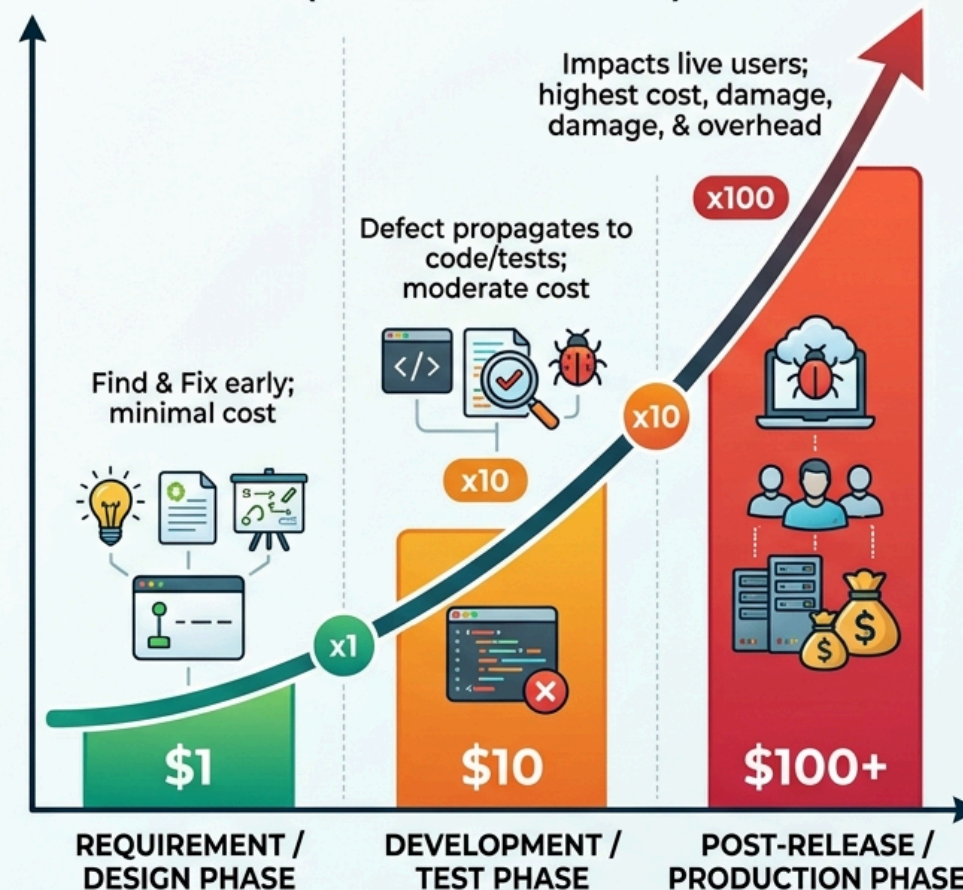
- **一致性成本 (Conformance Costs - 主動投資品質)：**
 - **預防成本 (Prevention)：**架構審查、合約設計、工程培訓與規範。
 - **評估成本 (Appraisal)：**單元測試、靜態程式碼分析 (SonarQube) 與 Code Review。
- **非一致性成本 (Non-Conformance Costs - 忽視品質的代價)：**
 - **內部失敗成本 (Internal Failure)：**上線前發現 Bug 的 Debug 與重測返工。
 - **外部失敗成本 (External Failure)：**生產環境崩潰、客戶求償、Hotfix 與商譽損失。
- **1:10:100 定律 (The Rule of Tens)：**
 - 需求階段修復缺陷代價 **\$1** → 開發測試階段暴增至 **\$10** → 上線後災難損失高達 **\$100 ~ \$1000+** !

品質成本 (CoQ) 架構與 1:10:100 定律

UNDERSTANDING COST OF QUALITY (CoQ)



THE 1:10:100 RULE OF TENS (Cost to Fix a Defect)



EARLY PREVENTION IS KEY: Prioritize Conformance to Avoid Escalating Non-Conformance Costs.

1.5 軟體生命週期中的品質把關

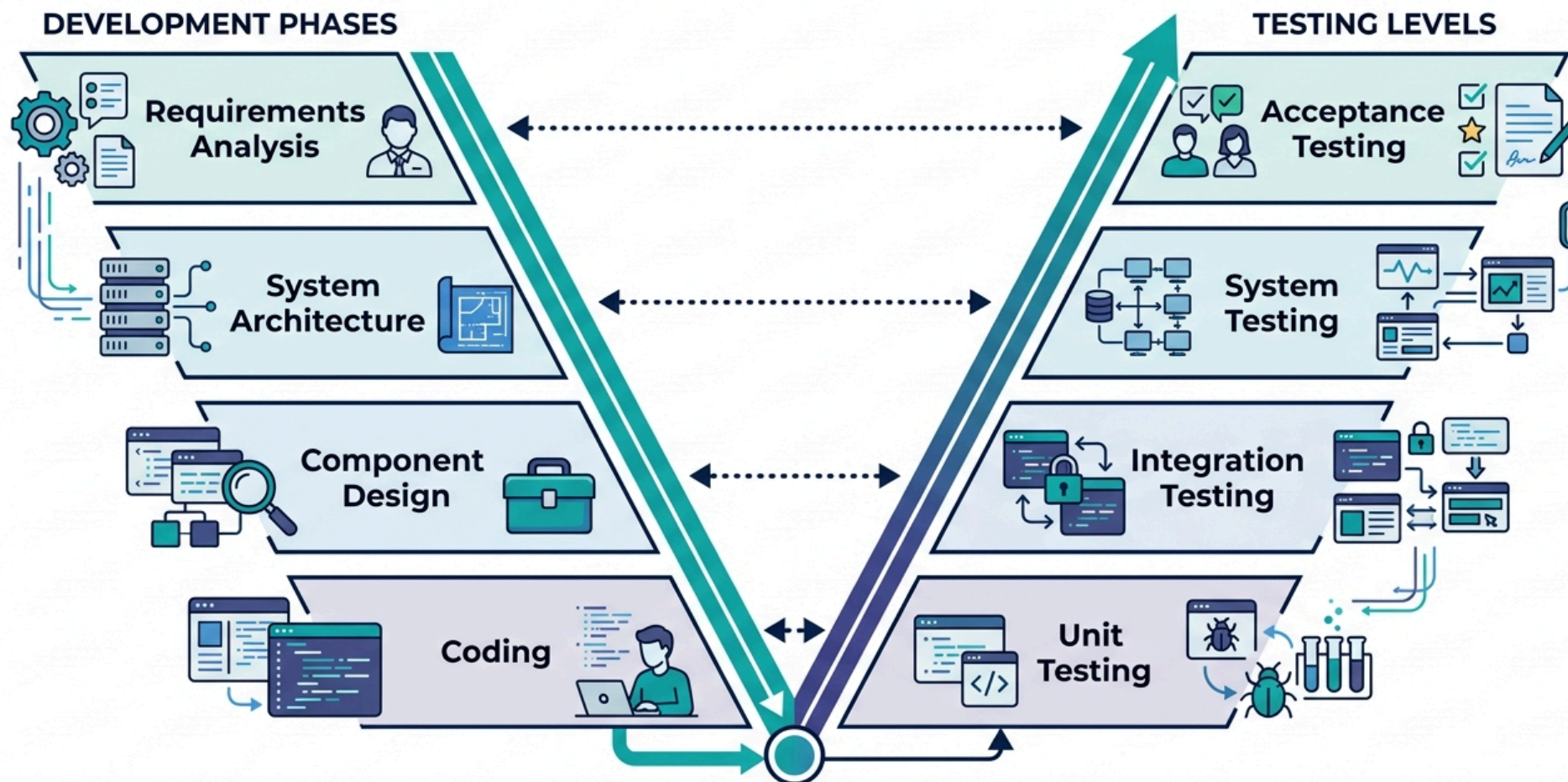
V 模型對稱性與 DevOps CI/CD 連續品質門檻

1.5.1 傳統模型與 V 模型：對稱性與早期規劃

- **V 模型 (V-Model)** 建立了開發階段與測試層級的嚴密對稱：
 - 需求分析 (Requirements) → 平行規劃 驗收測試 (Acceptance Testing)
 - 系統架構 (System Architecture) → 平行規劃 系統測試 (System Testing)
 - 元件設計 (Component Design) → 平行規劃 整合測試 (Integration Testing)
 - 編寫程式碼 (Coding) → 實作並執行 單元測試 (Unit Testing)
- **核心價值**：在寫下第一行業務程式碼前，各層級測試規格就已隨同架構確立。

V 模型 (V-Model) 開發與測試對稱圖

THE V-MODEL: INTEGRATING DEVELOPMENT AND TESTING

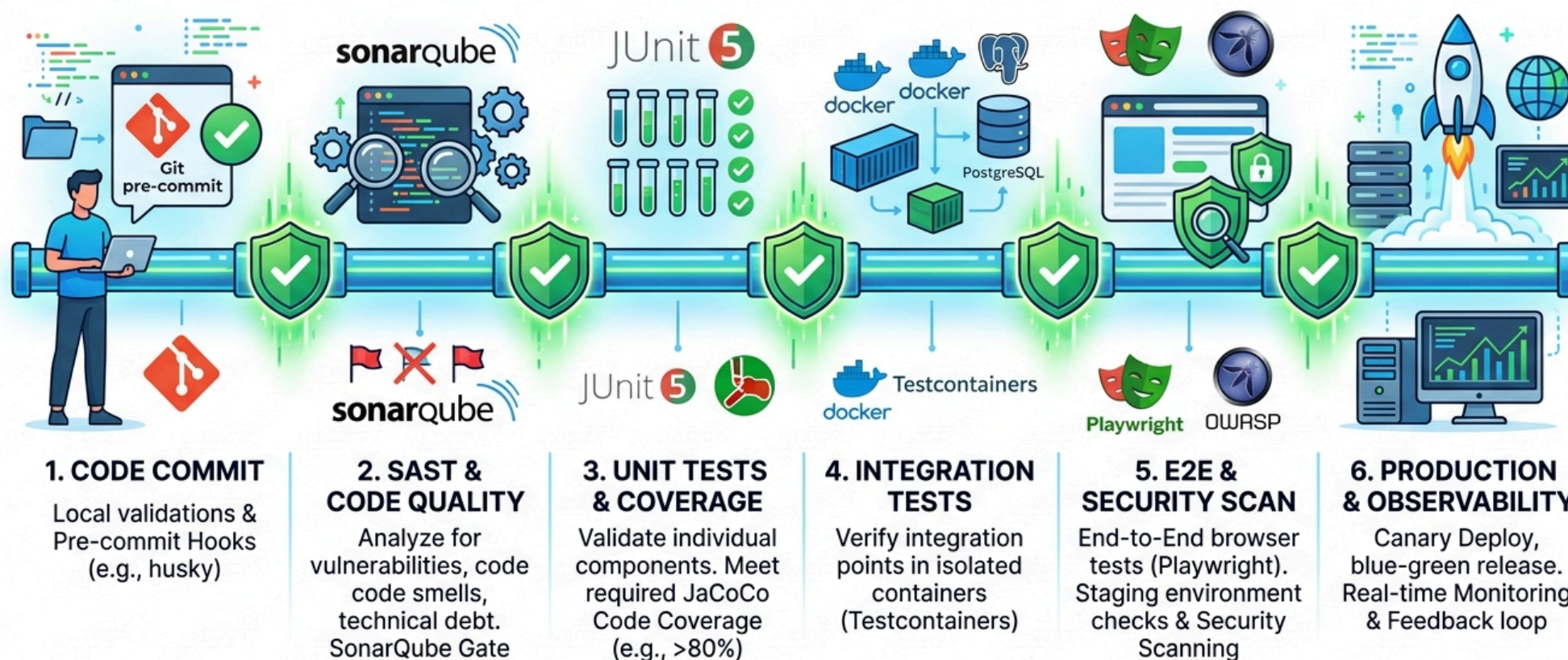


1.5.2 DevOps CI/CD 連續品質門檻 (Quality Gates)

- **1. Commit 門檻**：Git Pre-commit Hook 格式化與靜態語法檢查。
- **2. SAST 靜態代碼品質門檻**：SonarQube / SpotBugs 掃描代碼異味與安全弱點。
- **3. Unit Tests & 覆蓋率門檻**：JUnit 5 單元測試，JaCoCo 驗證覆蓋率 (> 80%)。
- **4. Integration Tests 容器門檻**：Testcontainers 拉起真實 Docker 驗證資料庫與 API。
- **5. E2E & Security 驗收門檻**：Playwright 自動化使用者流程 + OWASP ZAP 動態掃描。
- **6. Production 部署自愈門檻**：金絲雀/藍綠部署 + 可觀測性監控 P99 延遲告警。

DevOps CI/CD 6 大連續品質門檻架構圖

CONTINUOUS QUALITY GATES IN MODERN DEVOPS CI/CD PIPELINE



1.6 現代軟體品質模型 (ISO 25010)

ISO 25010 八大產品品質特性與 16 週實戰測試技術地圖

不同產業與產品具備截然不同的品質模型

DIFFERENT INDUSTRIES & PRODUCTS HAVE UNIQUE QUALITY MODELS



ISO 25010 八大產品品質特性架構

ISO 25010 PRODUCT QUALITY CHARACTERISTICS

(Focusing only the first level)



1. Functional Suitability

Meets stated and implied needs



2. Reliability

Maintains performance over time



3. Performance Efficiency

Optimizes response times and resources



4. Usability

Easy to learn, understand, and operate



5. Security

Protects information and assets



6. Maintainability

Can be modified, tested, or repaired efficiently



7. Portability

Adaptable to different environments



8. Compatibility

Interacts seamlessly with other systems

1.6.1 ISO 25010 特性解析 (1/2)

- **1. 功能適合性 (Functional Suitability) :**
 - 完備性 (Completeness)、正確性 (Correctness)、適切性 (Appropriateness)。
- **2. 可靠性 (Reliability) :**
 - 成熟度 (Maturity)、容錯度 (Fault Tolerance)、可回復性 (Recoverability)。
- **3. 效能效率 (Performance Efficiency) :**
 - 時間行為 (Time Behavior, P99 延遲)、資源利用率、容量 (Capacity)。
- **4. 易用性 (Usability) :**
 - 易識別性、易學習性、易操作性、使用者錯誤防護 (Error Protection)。

1.6.1 ISO 25010 特性解析 (2/2)

- 5. 安全性 (Security) :
 - 機密性 (Confidentiality)、完整性 (Integrity)、抗抵賴性 (Non-repudiation)、真實性與授權。
- 6. 可維護性 (Maintainability) :
 - 模組化 (Modularity)、可分析性、可修改性、可測試性 (Testability)。
- 7. 可移植性 (Portability) :
 - 適應性 (Adaptability)、易安裝性、易置換性 (Docker 容器化一致性)。
- 8. 相容性 (Compatibility) :
 - 共存性 (Co-existence)、互通性 (Interoperability, API/微服務契約)。

1.6.2 ISO 25010 與 16 週實戰技術地圖 (1/2)

ISO 25010 品質特性	核心子特性	本課程對應測試與工程技術
功能適合性 (Functional Suitability)	完備性、正確性、適切性	等價類分割 (EP)、邊界值分析 (BVA)、JUnit 5、BDD (Cucumber)
可靠性 (Reliability)	成熟度、容錯度、可回復性	斷言 (Assertions)、屬性測試 (jqwik Property-Based Testing)、混沌工程
可維護性 (Maintainability)	模組化、可分析性、 可測試性	靜態代碼分析 (SonarQube/SpotBugs)、變異測試 (PITest)、依賴解耦
安全性 (Security)	機密性、完整性、抗抵賴性	靜態安全掃描 (AST/SAST)、模糊測試 (Fuzzing with Jazzer)

1.6.2 ISO 25010 與 16 週實戰技術地圖 (2/2)

ISO 25010 品質特性	核心子特性	本課程對應測試與工程技術
效能效率 (Performance Efficiency)	時間行為 (延遲)、資源利用率	k6 / JMeter 高併發壓測、GC 監控與記憶體洩漏分析
相容性 (Compatibility)	共存性、互通性 (Interoperability)	微服務契約測試 (Pact) 、跨版本相容性測試
可移植性 (Portability)	適應性、易安裝性、易置換性	Testcontainers 容器化測試、雲原生多環境測試
易用性 (Usability)	易識別性、易學習性、錯誤保護	Playwright E2E 驗收測試、使用者流程自動化驗證

1.7 綜合練習與思維激盪

課堂思考與實務討論

1.7 課堂思維激盪與問題討論

- **1. AI 時代的品質反思：**

- 當生成式 AI 可在幾秒內產生程式碼時，為什麼軟體測試工程師的價值反而大幅提升？
- 請從「**Test Oracle 問題**」與「**自我印證偏誤**」兩方面進行思考。

- **2. ISO 25010 維度分析：**

- 「微服務系統在資料庫當機重啟後，能在 5 秒內自動重連並重試訊息，完全不丟失交易。」
- 這體現了 ISO 25010 中的哪些品質特性？（提示：容錯度、可回復性、資料完整性）。

- **3. 數值精度與累計誤差實證：**

- 連續將 0.1 累加 1,000,000 次，比較其結果與 100000.0 的偏差。